Parallel Optical Bit Pooling with Commit-Only Hidden-Number Recovery: A Probabilistic and Information-Theoretic Analysis

George Wagenknecht

13 November 2026

Abstract

We analyse a software bench in which a hidden integer N is rendered as a two-dimensional grid of black and white cells, observed through a camera, reduced by parallel spatial pooling, classified bit by bit, accumulated by majority vote, and finally accepted only if the reconstruction matches a published salted SHA-256 commitment. We formalise each stage. Spatial pooling is shown to be an orthogonal projection written as a Kronecker product, and under Gaussian sensor noise it yields an erasure-augmented binary symmetric channel (BSC) with a 256-fold variance reduction. Vote accumulation is analysed through a coupon-collector argument (the cover time is asymptotically Gumbel) and a Chernoff–Bhattacharyya bound with the closed form P(bit error) ⩽ (1 − r(1 − Z))^T, where r is the reveal rate, T the number of exposures, and Z = 2√(ε(1 − ε)). The decoder's bounded bit-flip search over the least-confident positions is identified as a Chase-type list decoder in which the hash commitment plays the role of an ideal error-detecting code. We derive its candidate budget, false-acceptance probability, and success condition, and we compare the required number of exposures with the Shannon limit of the channel. For the default instance N = 9 999 999 999 999 999 = $3^2 \cdot 11 \cdot 17 \cdot 73 \cdot 101 \cdot 137 \cdot 5\,882\,353$ (56 grid cells), we predict a mean cover time of 7.7 exposures and reliable recovery in about 20 exposures. We also analyse the auxiliary Qiskit backend, which prepares an 8-qubit GHZ state purely as an execution diagnostic. We state clearly that the system does not factor integers: the factors are held by the oracle and are released only after a successful commitment proof.

Keywords: spatial pooling, majority voting, Chernoff bound, Chase decoding, commitment scheme, binary symmetric channel, GHZ diagnostic.

1. Introduction

The bench studied here (`rp_hidden_number_commit_only.py`) separates three roles.

1. **Oracle.** It holds a secret integer N, a random 128-bit salt s, and the commitment c = SHA-256(s ‖ N), where N is encoded as a big-endian byte string of fixed length. The oracle also precomputes the prime factorisation of N. It publishes only (s, c).
2. **Optical channel.** The bits of N are rendered as a rows × cols grid of cells; a camera (or a synthetic source) produces a frame of that grid on each cycle.
3. **Recogniser.** It sees only (s, c) and the frames. It pools each frame in parallel, accumulates votes, and tests candidates against c. On success, the oracle reveals N and its factors.

The contribution of this paper is a mathematical account of why such a pipeline works, how many frames it needs, and what it does *not* accomplish. The framing is deliberately modest. The design is a noisy-channel recovery problem with a cryptographic verifier. It is not an attack on factoring, and the quantum component is a diagnostic only.

1.1 Notation

- m: number of grid cells (and of bits in the padded representation); m = rows · cols.
- b ∈ {0,1}^m: the oracle's bit vector, b_i = ⌊N / 2^i⌋ mod 2.
- r ∈ (0,1]: reveal probability (per cell, per cycle).
- ε ∈ [0, ½): bit-flip probability of the simulated sensor noise.
- T: number of cycles (exposures).
- τ: classification threshold (default 127); c_0: confidence threshold (default 25).
- $H_2(x) = -x \log_2 x - (1 - x) \log_2(1 - x)$: binary entropy.
- $Q(x) = 1 - \Phi(x)$: Gaussian tail.

---

2. Parallel Spatial Pooling as a Linear Operator

2.1 Definition

Let G ∈ ℝ^(16R × 16C) be the grayscale frame after resizing, with R = rows, C = cols. Each cell (i, j) is a 16 × 16 block. The pooled value is

$p_{ij} = (1/256) \sum_{u=0}^{15} \sum_{v=0}^{15} G(16i + u, 16j + v)$.

Define the averaging row vector $a = (1/16) \cdot 1^T \in \mathbb{R}^{1\times16}$. Then

$\mathrm{vec}(P) = (A_R \otimes A_C)\, \mathrm{vec}(G)$, with $A_k = I_k \otimes a$.

The operator $\Pi = (A_R \otimes A_C)$ has $\Pi \Pi^T = (1/256)\, I_{RC}$, so $\Pi^T \Pi / 1 \cdot 256$ is the orthogonal projector onto the subspace S of block-constant images. Pooling therefore keeps exactly the component of the frame that is constant on each cell and discards the rest. The

implementation realises this through a reshape to (R, 16, C, 16) followed by a mean over axes (1, 3), a single vectorised call. This is what "parallel" means in the code: all m cells are reduced by one tensor operation rather than by m sequential decisions.

2.2 Noise suppression

Suppose each pixel carries independent zero-mean noise of variance σ^2. Then

$\mathrm{Var}(p_{ij}) = \sigma^2 / 256$, so $s := \sigma / 16$.

The standard deviation falls by a factor of 16, which is a 24 dB signal-to-noise gain ($10 \log_{10} 256 \approx 24.1$ dB).

2.3 From pooled values to a channel

Let the nominal pooled levels be $\mu_0 \approx 0$ for a black cell and $\mu_1 \approx 255$ for a white cell. Write $d_1 = \mu_1 - \tau$ (≈ 128) for a white cell and $d_0 = \tau - \mu_0$ (≈ 127) for a black cell. For a white cell under Gaussian noise $p \sim \mathcal{N}(\mu_1, s^2)$:

- **Valid and wrong:** $p < \tau - c_0$, with probability $\varepsilon_{\mathrm{eff}} = Q((d_1 + c_0) / s)$.
- **Erased (invalid):** $|p - \tau| < c_0$, with probability $\rho = Q((d_1 - c_0) / s) - Q((d_1 + c_0) / s)$.
- **Valid and right:** otherwise.

The black-cell case is symmetric with d_0. The confidence gate thus converts an ordinary BSC into a **binary symmetric erasure channel** whose erasure rate ρ is traded against its error rate $\varepsilon_{\mathrm{eff}}$. Raising c_0 pushes $\varepsilon_{\mathrm{eff}}$ down at the price of more erasures. Because errors fall like $\exp(-(d + c_0)^2 / 2s^2)$, a modest c_0 often removes nearly all errors while costing few erasures when $s \ll d$.

2.4 Misalignment tolerance

Let the camera be displaced by a fraction β of a cell along one axis. A white cell bordering a black cell then has pooled value $\approx 255(1 - \beta)$, and a black cell bordering a white cell has value $\approx 255\beta$. The gate demands that both remain valid and correctly classified:

$255(1 - \beta) > \tau + c_0 = 152 \Rightarrow \beta < 0.404$, $255\,\beta < \tau - c_0 = 102 \Rightarrow \beta < 0.400$.

So the design tolerates a shift of about 40% of a cell width in the worst one-dimensional case, before any averaging benefit from noise. Larger geometric distortions (perspective, rotation) require a calibration step the present code does not implement.

2.5 What the synthetic mode actually models

In the code, pooling is followed by an injected XOR mask, observed ← observed ⊕ Bernoulli(ε), and an independent reveal mask Bernoulli(r) intersected with the validity mask. This is the idealised channel of Section 3, applied directly at the bit level. Physical

camera noise would produce the Gaussian erasure-error channel of Section 2.3 instead. In either case the downstream mathematics is identical once (r, ε) are replaced by effective values r_eff = r(1 − ρ) and ε_eff.

---

3. The Observation Channel

Fix a bit position i with true value b_i. In cycle t the recogniser receives

Y_{i,t} = ∅ with probability 1 − r, Y_{i,t} = b_i ⊕ Z_{i,t} with probability r, where Z_{i,t} ~ Bernoulli(ε),

independently across i and t. This is a memoryless binary symmetric erasure channel used T times on a **static** input, which is a repetition code of length T per bit. Its Shannon capacity per channel use is

$C = r\,(1 - H_2(\varepsilon))$.

For the defaults r = 0.45 and ε = 0.02, $H_2(0.02) \approx 0.1414$, hence C ≈ 0.386 bits per cell per frame.

Because the m cells are exposed simultaneously, one frame supplies m · C bits of mutual information about the m-bit secret. A converse argument gives the lower bound

T ⩾ m / (m C) = 1 / C ≈ 2.59 frames

for *any* decoder with vanishing error. We will see that the actual pipeline needs several times this number of frames, because it applies no coding across cells. The gap is the price of using an uncoded payload.

---

4. Vote Accumulation

4.1 Sufficient statistic

The recogniser maintains counters (ones_i, zeros_i). Define N_i = ones_i + zeros_i (the number of valid observations of cell i) and F_i, the number of those observations that were flipped by noise. Then N_i ~ Bin(T, r) and, given N_i, F_i ~ Bin(N_i, ε). Since the likelihood ratio is

Λ_i = log [P(Y | b_i = 1) / P(Y | b_i = 0)] = (ones_i − zeros_i) · ln((1 − ε) / ε),

the difference D_i = ones_i − zeros_i is a **sufficient statistic** and majority voting is exactly the maximum-likelihood decision for each bit. The per-bit posterior log-odds is proportional to D_i, so the code's confidence |ones_i − zeros_i| is the posterior reliability up to the constant ln((1 − ε)/ε).

4.2 Cover time

The decoder (`_decode`) refuses to act until every cell has been observed at least once. A given cell remains unseen after T cycles with probability (1 − r)^T. Let T_cov be the first cycle at which all m cells have been seen. With λ := −ln(1 − r),

P(T_cov > T) = 1 − (1 − e^{−λT})^m ⩽ m e^{−λT},

and asymptotically in m,

λ T_cov − ln m ⇒ Gumbel(0, 1),

so

E[T_cov] ≈ (ln m + γ) / λ, Var(T_cov) ≈ π² / (6 λ²),

where γ ≈ 0.5772 is the Euler–Mascheroni constant. This is the classical coupon-collector law (Erdős and Rényi, 1961). For m = 56 and r = 0.45: λ = 0.5978, E[T_cov] ≈ (4.025 + 0.577)/0.5978 ≈ 7.7 cycles, with standard deviation ≈ 1.2825/0.5978 ≈ 2.2 cycles.

4.3 Majority-vote error

For a transmitted 1 the vote fails when F_i ⩾ N_i / 2 (ties decode to 0, because the code tests ones > zeros). For a transmitted 0 it fails when F_i > N_i / 2. In both cases the Chernoff bound for a Binomial(N, ε) tail gives

P(error | N_i = n) ⩽ exp(−n · D_KL(½ ‖ ε)) = (4ε(1 − ε))^{n/2} = Z^n,

where Z := 2√(ε(1 − ε)) is the Bhattacharyya parameter of the BSC. The exponent D_KL(½ ‖ ε) = −ln Z is the true error exponent for per-bit majority decoding (the bound is tight on the exponential scale).

Marginalising over N_i ~ Bin(T, r) uses the probability generating function E[z^N] = (1 − r + r z)^T:

p_e(T) := P(bit i wrong after T cycles) ⩽ (1 − r(1 − Z))^T.

This closed form shows that the decoder's reliability improves geometrically with base 1 − r(1 − Z). With the defaults, Z = 2√(0.02 · 0.98) = 0.280, so 1 − r(1 − Z) = 1 − 0.45 · 0.72 = 0.676 and −ln(0.676) = 0.392 nats per cycle.

4.4 Whole-word error and number of cycles

By the union bound, the probability that plain majority voting (no search) returns a wrong integer is

P(word error) ⩽ m (1 − r(1 − Z))^T.

To achieve P(word error) ⩽ δ, it suffices that

$T \geqslant \ln(m/\delta) / (-\ln(1 - r(1 - Z)))$.

For $m = 56$ and $\delta = 10^{-2}$ this gives $T \geqslant 8.63 / 0.392 \approx 22.0$; for $\delta = 10^{-6}$ it gives $T \geqslant 17.84 / 0.392 \approx 45.6$. Note that the dominant requirement is the error exponent, not the cover time (7.7 cycles), since a cell may be seen but wrongly classified.

---

5. Commitment-Verified List Decoding

5.1 The commitment as an error-detecting code

Pure majority decoding produces a single estimate $\hat{b}$. The recogniser adds an exact verifier $V(x) = [\mathrm{SHA\text{-}256}(s \,\|\, x) = c]$. Under the standard assumption that SHA-256 behaves as a random oracle with 256-bit output, a wrong candidate $x \neq N$ passes V with probability 2^{-256}. If the decoder tests Q candidates in total, the false-acceptance probability is at most

$P_{FA} \leqslant Q \cdot 2^{-256}$.

With the default budget $Q_{max} = 200\,000$, $P_{FA} \leqslant 1.73 \times 10^{-72} \cdot 10^{-5} \approx 2 \times 10^{-72}$, negligible. Thus the hash serves as a *perfect* error-detection channel: it tells the decoder with overwhelming reliability whether a candidate is correct, even though it carries no soft information.

5.2 Bounded bit-flip search (Chase-type decoding)

When $\hat{b}$ fails the check, the code ranks positions by $|D_i|$ (ascending), keeps the $L = \min(16, m)$ least-reliable positions U, and tests every candidate obtained by flipping at most $t = \min(4, L)$ bits inside U. This is a Chase-style algorithm (Chase, 1972) with test patterns confined to the L least-reliable positions. The number of candidates per cycle is

$V(L, t) = \Sigma_{k=0}^{t} C(L, k) = 1 + 16 + 120 + 560 + 1820 = 2517$.

Hence the hard limit $Q_{max} = 200\,000$ is exhausted after about $\lceil 200\,000 / 2517 \rceil \approx 80$ cycles in which the decoder is active. (The check against Q_{max} is applied inside the inner loop; the zero-flip test is always performed.)

5.3 Success condition

Let K be the number of erroneous bits of $\hat{b}$ and E the set of their positions. The search succeeds in a cycle exactly when

$K \leqslant t$ and $E \subseteq U$.

If bit errors were independent with probability p_e, then $K \sim \mathrm{Bin}(m, p_e)$ and

$P(K \leqslant t) = \Sigma_{k=0}^{t} C(m, k)\, p_e^k\, (1 - p_e)^{m - k}$.

The second condition ($E \subseteq U$) is favoured by the ranking: an erroneous bit has $F_i \geqslant N_i/2$, hence a small margin $|D_i| = |N_i - 2F_i|$, and therefore appears near the front of the ascending order. A rigorous bound would require the joint law of margins; the code does not provide it, and we do not claim one here.

A useful Poisson approximation uses $\mu = m \cdot p_e$. With $T = 20$, the bound of Section 4.3 gives $p_e \leqslant 0.676^{20} \approx 3.99 \times 10^{-4}$, so $\mu \leqslant 0.0223$ and $P(K \geqslant 5) \approx \mu^5/5! \approx 4.5 \times 10^{-12}$. In practice list decoding should succeed far earlier than $T = 20$, because it tolerates up to four wrong bits; however, the cover time (≈ 7.7 cycles) remains a hard lower floor, since the decoder cannot act while any cell is unknown.

5.4 Comparison with brute-force inversion

Without the optical channel, the recogniser would have to search the full space of 2^{56} candidates. Expected effort is $2^{55} \approx 3.6 \times 10^{16}$ hash evaluations. The channel reduces this to at most about 2.5×10^3 per cycle. The reduction arises because the observation channel supplies roughly $m \cdot C \approx 21.6$ bits of information per frame about the secret; the hash only *verifies*.

5.5 Why this is not a hiding commitment

Because the salt is published and N is a low-entropy 54-bit integer, the commitment does not hide N against an offline adversary who is willing to evaluate SHA-256 on 2^{54} candidates. The commitment is **binding** (collision resistance) but its *hiding* property depends on N having high min-entropy. In this bench the hash is used solely as a proof-of-reconstruction device, not as a secrecy mechanism.

5.6 Possible refinement: best-first enumeration

The code enumerates flip sets in order of cardinality k (and within a set, lexicographically). Since the posterior log-odds satisfy $|\Lambda_i| \propto |D_i|$, the posterior cost of flipping a set S is

$$W(S) = \Sigma_{i \in S} |D_i| \cdot \ln((1 - \varepsilon)/\varepsilon),$$

and maximum-a-posteriori decoding corresponds to testing sets in increasing $W(S)$. A priority-queue (best-first) enumeration would test the most probable candidates first and reduce the expected number of hashes at no cost in correctness. This is a proposed improvement, not a feature of the current code.

---

6. The Oracle and the Role of Factorisation

The oracle precomputes the prime factors of N. For the default $N = 10^{16} - 1$, the identity $(10^8 - 1)(10^8 + 1)$ gives

$N = 3^2 \cdot 11 \cdot 73 \cdot 101 \cdot 137 \cdot 17 \cdot 5\,882\,353$.

Here $10^8 - 1 = 3^2 \cdot 11 \cdot 73 \cdot 101 \cdot 137$ and $10^8 + 1 = 17 \cdot 5\,882\,353$. The bit length of N is 54, which the code rounds up to 56 bits (byte alignment), giving the grid 8 × 7.

The recogniser **never factors N**. The factors are computed by the oracle (via SymPy, with a trial-division fallback) and are returned only after `reveal_if_proven` re-verifies the candidate against the commitment. What the experiment demonstrates is *transmission of N through an optical channel*, authenticated by a hash. Any claim of factorisation by this system would be unfounded; the factorisation cost remains that of the best classical algorithms (for example, the number field sieve, with heuristic complexity exp((c + o(1)) (ln N)^{1/3} (ln ln N)^{2/3}), c = (64/9)^{1/3}).

---

7. The Diagnostic Quantum Backend

The script embeds an 8-qubit circuit and a custom `BackendV2` that simulates it with a dense state vector. The circuit applies a Hadamard to qubit 0 followed by a CNOT chain, preparing the GHZ state (Greenberger, Horne and Zeilinger, 1989):

$|\psi\rangle = U\,|0\rangle^{\otimes 8} = (\mathrm{CNOT}_{6,7} \cdots \mathrm{CNOT}_{0,1})(H \otimes I^{\otimes 7})\,|0\rangle^{\otimes 8} = (|0\rangle^{\otimes 8} + |1\rangle^{\otimes 8}) / \sqrt{2}$.

In the stabiliser formalism (Gottesman, 1997), $|\psi\rangle$ is the unique +1 eigenstate of the group generated by $X^{\otimes 8}$ and $Z_i Z_{i+1}$ for $i = 0, \ldots, 6$. Measurement in the computational basis returns the all-zeros or all-ones string, each with probability ½. With 64 shots, the count of the all-zeros outcome is Bin(64, ½), with mean 32 and standard deviation 4. This gives the diagnostic a testable signature: any other outcome string, or a strongly unbalanced split, indicates a defect in the backend.

The simulator stores $2^8 = 256$ complex amplitudes and applies each gate by an $O(2^n)$ loop; one circuit costs $O(n \cdot 2^n)$ operations. Sampling uses a categorical distribution over the 256 basis states. The gate loops handle only `h` and `cx`; the measurement instructions are ignored and the sampled bit strings are all that is returned.

Two observations are essential for scientific honesty:

1. **The result of the quantum diagnostic is discarded.** In `observe`, the output of `get_counts()` is assigned to a throwaway variable. The diagnostic cannot influence the decoded integer.

2. **No quantum speed-up is present.** The state is classically simulated, the circuit does not depend on N, and nothing in the circuit encodes a period-finding or phase-estimation problem. In particular, it is not Shor's algorithm.

---

8. Numerical Instantiation

For the default parameters ($m = 56$, $r = 0.45$, $\varepsilon = 0.02$, $t = 4$, $L = 16$, $Q_{max} = 2 \times 10^5$), the preceding sections predict the following values.

Quantity	Formula	Value
Bhattacharyya parameter Z	$2\sqrt{(\varepsilon(1-\varepsilon))}$	0.280
Per-cycle error base	$1 - r(1 - Z)$	0.676
Per-cycle exponent	$-\ln(1 - r(1 - Z))$	0.392
Channel capacity C	$r(1 - H_2(\varepsilon))$	0.386 bits
Shannon lower bound on T	$1/C$	2.59 frames
Mean cover time	$(\ln m + \gamma)/\lambda$	7.7 cycles
Cover-time s.d.	$\pi/(\sqrt{6}\,\lambda)$	2.2 cycles
T for union bound $\leqslant 10^{-2}$	$\ln(m/\delta)/0.392$	22.0
T for union bound $\leqslant 10^{-6}$	$\ln(m/\delta)/0.392$	45.6
Hamming-ball size V(16, 4)	$\Sigma\, C(16, k), k \leqslant 4$	2517
Cycles until Q_{max} reached	$\lceil Q_{max} / V \rceil$	≈ 80
False-accept bound	$Q_{max} \cdot 2^{-256}$	$\approx 2 \times 10^{-72}$

The design is thus comfortably inside its default 800-cycle budget. The Chase-style search lowers the practical requirement from roughly 22–46 cycles (plain majority) to a figure close to the cover time plus a few cycles, which is the regime in which the synthetic mode succeeds in practice.

For a physical camera, the parameters (r, ε) are replaced by $r_{eff} = r(1 - \rho)$ and ε_{eff} of Section 2.3. As a worked example, suppose $\sigma = 40$ grey levels per pixel, so $s = 2.5$. With $c_0 = 25$ and $d \approx 128$ we have $(d + c_0)/s = 61$, so ε_{eff} is numerically zero and $\rho \approx 0$. Pooling alone makes the channel effectively noiseless; in that regime the limiting factors are geometric (Section 2.4), not statistical.

---

9. Limitations and Threats to Validity

1. **Synthetic noise is not camera noise.** The noise model is a bit-flip applied after pooling; it ignores blur, rolling shutter, moiré between the display and the sensor, exposure variation, and Bayer demosaicing. Section 2.3 provides a better-justified model but is not what the code runs.
2. **No calibration.** The code assumes the grid fills the frame; the resize step assumes alignment. Real use needs corner detection or fiducials.
3. **Unused parameter.** The oracle's `noise_probability` argument is stored but never used; noise is applied by the main loop.
4. **Independence assumption.** The Chernoff analysis assumes independent noise across cycles. Fixed-pattern sensor noise and persistent glare violate this, and then repeated observation does not reduce the error.
5. **Deterministic tie-breaking.** `np.argsort` breaks equal-confidence ties by index, so the uncertain set U is biased toward low-index (least-significant) bits. Randomised tie-breaking would remove the bias.
6. **Quantum layer.** It is a diagnostic only (Section 7) and gives no computational advantage.
7. **Hiding.** The commitment does not hide low-entropy secrets (Section 5.5).

---

10. Conclusion

We gave a quantitative account of a parallel optical pipeline for recovering a hidden integer from noisy observations, with acceptance gated by a salted SHA-256 commitment. The main results are: (i) pooling is an orthogonal Kronecker projection with 256-fold variance reduction, inducing a binary symmetric erasure channel; (ii) cover time follows a Gumbel law with mean $(\ln m + \gamma)/(-\ln(1 - r))$; (iii) majority-vote error per bit is bounded by $(1 - r(1 - Z))^T$ with exponent $-\ln Z$, which is exponentially tight; (iv) the bounded bit-flip search is a Chase-type list decoder whose verifier is a hash, with total false-acceptance probability below $Q \cdot 2^{-256}$; and (v) the channel capacity $r(1 - H_2(\varepsilon))$ shows that uncoded repetition leaves a gap of about one order of magnitude relative to the information-theoretic limit on the number of frames. The quantum and factorisation aspects of the code are, by construction, ancillary: the system recovers a number that is transmitted to it, and the factors are supplied by the oracle.

Future work includes best-first candidate enumeration (Section 5.6), an error-correcting code across cells (for example, a Reed–Solomon or LDPC layer) to approach the Shannon limit, calibration for real cameras, and replacing the unused diagnostic with a benchmark that actually exercises the pooled data.

References

1. C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, 27, 1948.
2. H. Chernoff, "A Measure of Asymptotic Efficiency for Tests of a Hypothesis Based on the Sum of Observations," *Annals of Mathematical Statistics*, 23(4), 1952.
3. P. Erdős and A. Rényi, "On a Classical Problem of Probability Theory," *Magyar Tudományos Akadémia Matematikai Kutató Intézetének Közleményei*, 6, 1961.
4. D. Chase, "A Class of Algorithms for Decoding Block Codes With Channel Measurement Information," *IEEE Transactions on Information Theory*, 18(1), 1972.
5. D. M. Greenberger, M. A. Horne and A. Zeilinger, "Going Beyond Bell's Theorem," in *Bell's Theorem, Quantum Theory and Conceptions of the Universe*, Kluwer, 1989.
6. D. Gottesman, "Stabilizer Codes and Quantum Error Correction," PhD thesis, Caltech, 1997.
7. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006.
8. National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.
9. A. K. Lenstra and H. W. Lenstra Jr. (eds.), *The Development of the Number Field Sieve*, Lecture Notes in Mathematics 1554, Springer, 1993.
10. Qiskit contributors, *Qiskit: An Open-Source Framework for Quantum Computing*, https://qiskit.org.